

Bridging the Semantic-Action Gap in Visual Token Pruning for Efficient VLA Inference

Ziyan Liu^{1,2*} Yeqiu Chen^{1,2*} Yiming Zhang¹ Hongyi Cai¹ Tao Lin¹
Runquan Gui² Shuo Yang³ Zheng Liu⁴ Bo Zhao^{1†}

¹School of AI, Shanghai Jiao Tong University

²University of Science and Technology of China

³Harbin Institute of Technology (Shenzhen)

⁴BAAI

Abstract

Vision-Language-Action (VLA) models have shown great potential for embodied AI by integrating visual perception, language understanding, and action execution. In real-time deployment, these models must process continuous visual streams, incurring substantial computational overhead. Visual token pruning—a mainstream technique for accelerating Vision-Language Models (VLMs) by retaining salient tokens while discarding redundant ones—offers a natural candidate solution to this challenge. However, directly applying VLM-oriented pruning methods to VLA inference can cause severe degradation in manipulation performance. Our analysis attributes this degradation to a key mismatch: VLA inference exhibits distinct attention patterns between the vision-language prefill stage and the action-decode stage, so pruning based only on context-prefill *semantic salience* is biased toward semantic cues and may remove action-critical visual tokens. Motivated by this observation, we propose **VLA-Pruner**, an effective plug-and-play token pruning method grounded in the visual requirements of VLA inference, further exploiting the temporal continuity of robot manipulation. Specifically, VLA-Pruner estimates visual-token importance from both semantic prefilling and temporally smoothed action relevance, and then applies a Combine-then-Filter strategy to retain compact, non-redundant tokens under the compute budget. Experiments show that VLA-Pruner outperforms state-of-the-art approaches across multiple VLA architectures, achieving up to 1.99× speedup with comparable manipulation quality. We release our code at <https://github.com/MINT-SJTU/VLA-Pruner>.

1 Introduction

Recent advances in robot learning have led to remarkable progress in developing generalist policies capable of acting across diverse tasks and environments (Ma et al., 2024; Firoozi et al., 2025; Zhao et al., 2025; Brohan et al., 2022; Zitkovich et al., 2023; Kim et al., 2024; Black et al., 2024; Li et al., 2024a; Chi et al., 2025). Vision-Language-Action (VLA) models (Brohan et al., 2022; Zitkovich et al., 2023; Kim et al., 2024; 2025a; Black et al., 2024; Li et al., 2024a; Lin et al., 2025a), which benefit from the multimodal understanding and transfer ability of Vision-Language Models (VLMs) (Liu et al., 2023b; Bai et al., 2023), have emerged as a promising pathway toward general-purpose robot policy. Leveraging large-scale real-world robotic datasets (O’Neill et al., 2024), VLA models can perceive visual scenes, understand language instructions, and execute low-level actions across

*Equal contribution.

†Corresponding author: bo.zhao@sjtu.edu.cn.

diverse scenarios. However, real-time deployment requires VLA models to process continuous visual streams in dynamic environments, incurring heavy computational overhead. Existing VLA acceleration techniques, such as model lightweighting (Wen et al., 2025), quantization (Park et al., 2024), and early-exit (Yue et al., 2024), mitigate this cost but often require architectural modifications or retraining, limiting their generalizability.

A particularly appealing direction for training-free VLA acceleration is to reduce the computation induced by visual tokens. Prior VLM studies have shown that visual tokens are highly redundant (Chen et al., 2024; Shang et al., 2024; Alvar et al., 2025), and visual token pruning, which retains the most salient tokens while pruning the rest, has become a general and effective approach to improve VLM inference efficiency (Alvar et al., 2025; Chen et al., 2024; Zhang et al., 2024; Shang et al., 2024; Ye et al., 2025; Xing et al., 2024; Zou et al.). Since modern VLA models typically build on VLM backbones and inherit their visual-token processing pipeline, visual token pruning shows promise for VLA acceleration. Existing visual token pruning methods typically define **importance criteria** for tokens, such as attention scores (Chen et al., 2024; Zhang et al., 2024; Zou et al.), gradient information (Mao et al., 2025b;a) or feature diversity (Alvar et al., 2025) to quantify the significance of visual tokens, and less important tokens are pruned at inference time. By reducing visual redundancy, they yield substantial compute savings with minimal accuracy loss on standard VLM tasks such as image captioning (Agrawal et al., 2019; Plummer et al., 2015), VQA (Hudson and Manning, 2019; Liu et al., 2024), and video understanding (Xu et al., 2017; Li et al., 2016).

Despite recent progress, directly applying VLM-oriented visual token pruning methods to VLA inference can lead to severe degradation in manipulation performance, especially at high pruning ratios (see Figure 1), as also observed in prior work (Xu et al., 2025). We analyze the source of this degradation and identify a key mismatch: VLA inference involves both high-level semantic understanding and task planning, as well as low-level action execution, which induce distinct visual requirements across the **vision-language prefill** and **action decode** stages. Correspondingly, we observe markedly different attention patterns between these two stages: prefill attention tends to reflect semantic relevance in the vision-language context, whereas action-decode attention captures visual information needed for precise action generation. However, existing VLM-oriented pruning methods typically rank visual tokens using *semantic salience* measured during context prefilling—e.g., prefill attention in FastV (Chen et al., 2024) or text-to-vision cross-attention scores in SparseVLM (Zhang et al., 2024). Such semantic-only criteria bias token retention toward tokens salient for prefilling, thereby pruning action-critical visual tokens and limiting their applicability to VLA models.

To bridge this gap, we propose **VLA-Pruner**, a plug-and-play token pruning approach that accounts for stage-wise visual requirements and exploits the temporal continuity of robot manipulation. First, it performs **semantic-action importance estimation** for visual token retention: vision-language prefill attention scores measure semantic relevance, following prior VLM pruning works (Chen et al., 2024; Ye et al., 2025; Mao et al., 2025b; Zhang et al., 2024), while action-to-vision attention scores measure action relevance. Since current action-to-vision attention is unavailable when pruning is performed during prefilling, VLA-Pruner estimates it from recent decode attentions using a temporal smoother, motivated by the strong short-term overlap of top-attended visual patches across consecutive control steps. Second, VLA-Pruner adopts a **Combine-then-Filter token selection strategy** to avoid fragile score fusion. It first takes the union of tokens salient for either semantic prefilling or action decoding to maximize relevance, and then filters redundant candidates to obtain a compact token set under the given compute budget. This design reduces computational overhead while preserving visual information needed for both semantic understanding and action execution.

We summarize our contributions. (1) We systematically analyze visual-token importance in VLA inference, revealing distinct attention patterns between semantic prefilling and action decoding as well as temporal consistency in action-to-vision attention, thereby identifying the limitations of current

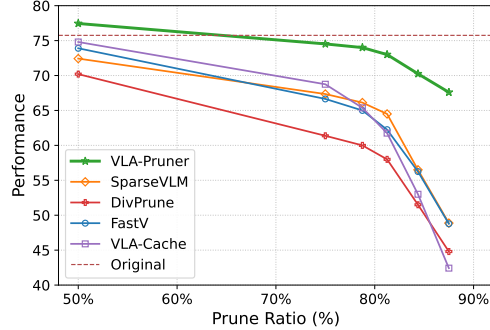


Figure 1: Comparison of visual token pruning/caching methods across various pruning ratios. Directly applying VLM pruning methods to VLA inference causes severe performance degradation.

VLM pruning methods for VLA acceleration. (2) We propose VLA-Pruner, a general, training-free framework that combines semantic-action importance estimation with a Combine-then-Filter token selection strategy, exploiting temporal continuity to preserve action-relevant visual information. (3) As a plug-and-play module for multiple VLA models, VLA-Pruner achieves state-of-the-art performance across two simulated environments, delivering up to $1.99\times$ speedup with negligible performance drops and remaining robust even at an 87.5% pruning ratio. We also validate its real-world advantage on a 6-DoF xArm6 robot.

2 Preliminary

VLA Inference. Vision-Language-Action (VLA) models process sequential inputs of the form $\{V^t, L^t\}$ into A^t across timesteps t . V^t represents the input visual observations, L^t denotes language task instructions, and $A^t = \{a_1^t, \dots, a_{\hat{N}}^t\}$ denotes the predicted action tokens, which are detokenized into executable robotic actions. The text input is tokenized into N text tokens $\mathbf{E}_\tau^t = \{\tau_1^t, \dots, \tau_N^t\}$. The vision encoder processes V^t into image features, which are projected into M visual tokens $\mathbf{E}_v^t = \{v_1, \dots, v_M\}$. These tokens, along with a proprioceptive token representing robot state, form the Transformer input. Since self-attention scales quadratically with token counts, the M (generally $M \gg N$) visual tokens dominate inference cost.

VLA Attention Breakdown. Like VLMs, VLA models rely on the attention mechanism of transformer for token interactions. Without loss of generality, we describe the single-head attention below. At each layer l , the model takes previous-layer hidden states of visual tokens $\mathbf{H}_v^{l-1} \in \mathbb{R}^{M \times d}$ and textual tokens $\mathbf{H}_\tau^{l-1} \in \mathbb{R}^{N \times d}$ as inputs. *For mathematical simplicity, we omit time/layer superscripts and treat the proprioceptive token as part of the vision-language context.* These inputs are transformed into queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$ using linear projections. Then, the shared vision-language context for attention and interaction is formed as: $\mathbf{K}_{\text{vl}} \in \mathbb{R}^{(N+M) \times d_k}$, $\mathbf{V}_{\text{vl}} \in \mathbb{R}^{(N+M) \times d_v}$. We break down VLA inference into two stages:

(1) Vision-Language prefill Stage. In this stage, queries come from the text and vision tokens: $\mathbf{Q}_{\text{vl}} \in \mathbb{R}^{(N+M) \times d_k}$. The attention matrix is computed by

$$\mathcal{A}_{\text{vl}} = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{vl}}\mathbf{K}_{\text{vl}}^\top}{\sqrt{d_k}}\right). \quad (1)$$

Attention score for each input visual patch is computed as

$$\mathcal{S}_{\text{vl}}[m] = \frac{1}{M+N} \sum_{i=1}^{M+N} \mathcal{A}_{\text{vl}}[i, m], \quad m = 1, \dots, M. \quad (2)$$

(2) Action decode Stage. In this stage, queries are action tokens $\mathbf{Q}_{\text{act}} \in \mathbb{R}^{K \times d_k}$ ($K = 1$ for autoregressive (Kim et al., 2024), $K = \hat{N}$ for chunk-based (Black et al., 2024)). Attention is computed as

$$\mathcal{A}_{\text{act}} = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{act}}\mathbf{K}_{\text{vl}}^\top}{\sqrt{d_k}}\right). \quad (3)$$

For the autoregressive case, attention vectors from $\hat{N}$ decoding steps are concatenated into the final attention matrix. The action-to-vision attention score in this stage is

$$\mathcal{S}_{\text{act}}[m] = \frac{1}{\hat{N}} \sum_{i=1}^{\hat{N}} \mathcal{A}_{\text{act}}[i, m], \quad m = 1, \dots, M. \quad (4)$$

3 Analyzing Visual Token Importance in VLA Inference

We systematically delve into the intrinsic characteristics of VLA inference (Figure 2). Our analysis proceeds in three stages. We first show that visual-token prefilling dominates the computational cost of VLA inference, making token reduction a direct route to acceleration (§3.1). We then reveal distinct attention patterns between semantic prefilling and action decoding, exposing the bias of semantic-only pruning criteria (§3.2). Finally, we show that action-to-vision attention remains temporally consistent across consecutive control steps, enabling recent decode attention to approximate current action-level

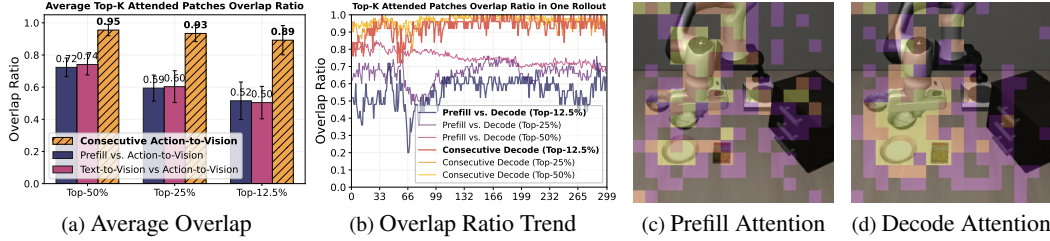


Figure 2: **Distinct attention patterns across VLA inference.** (a–b) Overlap ratios of Top- k attended patches among vision–language prefill, text-to-vision attention, action decode, and consecutive action-decode timesteps. We report average ratios (a) and one rollout trend (b). (c–d) Prefill (c) and action-decode (d) attention on the same frame, with top 12.5% (yellow), 25% (orange), and 50% (purple) patches overlaid. Prefill attention shows broad semantic coverage, whereas action decoding is locally focused. These results reveal stage-wise visual-token importance mismatch in VLA inference.

token importance (§3.3). All analyses are conducted on OpenVLA (Kim et al., 2024) with the LIBERO benchmark (Liu et al., 2023a). Together, these findings reveal a stage-wise mismatch in visual token importance: tokens that are salient for semantic prefilling are not necessarily sufficient for action decoding. Our findings identify the fundamental limitations of current VLM-specific token pruning methods for VLA models, which explains the performance degradation of semantic-only pruning methods and provides the empirical basis for VLA-Pruner.

3.1 Visual-Token Prefilling as the Computational Bottleneck

We first characterize the computational bottleneck of VLA inference. As noted in prior VLM studies (Zhang et al., 2024; Chen et al., 2024), incorporating visual tokens inevitably introduces substantial memory and computational overhead. This overhead is further amplified in VLA models, which repeatedly process dense visual observations at every control step. In typical VLA settings, each frame typically contains $256 \times n$ visual tokens, where n denotes the number of camera views, often an order of magnitude more than text tokens (~ 30 – 50) or action tokens (~ 7 – 56). Thus, processing the input visual stream becomes a primary contributor to inference cost. Since visual representations are highly redundant (Alvar et al., 2025; Chen et al., 2024; Zhang et al., 2024), visual token pruning shows great promise for VLA acceleration.

3.2 Stage-wise Attention Divergence in VLA Inference

Analysis Setup. VLA models integrate high-level semantic understanding and task planning with low-level action execution (Firoozi et al., 2025; Han et al., 2024; Huang et al., 2025; Sloman, 1996; Cui et al., 2025). We examine whether these two functional requirements are reflected in the attention behavior of VLA models. Specifically, we compare visual token attention in two stages of VLA inference: the vision-language prefill stage and the action decode stage. The prefill stage uses visual and language tokens to build the vision-language context, while the action-decode stage queries this context to generate executable actions. Attention scores are averaged across Transformer layers unless otherwise specified.

Results. Figure 2 reveals a clear divergence between the two stages. Prefill attention exhibits a broad semantic distribution, while action-decode attention shows localized focus essential for precise motor control, as exemplified in Figure 2c and Figure 2d. Quantitatively, the overlap ratio of top-attended visual tokens between vision-language prefilling and action decoding averages $\sim 50\%$ and often falls below 30% in individual rollouts (Figure 2). This indicates that action decoding relies on a set of visual tokens that partially coincides with the tokens salient during semantic prefilling.

Key Finding. This stage-wise attention divergence exposes a fundamental limitation of VLM-oriented pruning methods. Existing methods such as FastV (Chen et al., 2024) and SparseVLM (Zhang et al., 2024) typically rank visual tokens using semantic salience measured during context prefilling, such as prefill attention or text-to-vision attention. Such criteria can be effective for standard VLM tasks, but they are biased toward tokens useful for semantic prefilling. In VLA inference, this bias may prematurely remove tokens that are less salient during prefilling but crucial for action decoding. This failure mode becomes especially harmful at high pruning ratios, since removing even a small

number of high-importance tokens can substantially degrade model performance (Zhang et al., 2023; Ge et al., 2023). A qualitative example is provided in Figure 6.

3.3 Temporal Continuity of Action-to-Vision Attention

Practical Challenge. The above analysis suggests that robust VLA token pruning should preserve both semantic-level and action-level visual information. However, pruning must be performed during early-layer prefilling to reduce the dominant visual-token computation (Chen et al., 2024). At this point, the current action-decode attention is not yet available. This creates a practical tension: action-level visual importance is necessary for reliable token selection, but it cannot be directly observed when pruning decisions are made.

Temporal Analysis. VLA robot manipulation exhibits *temporal continuity* (Xu et al., 2025): visual observations and action targets typically evolve smoothly over short horizons. We therefore analyze the overlap between top-attended visual tokens in consecutive action-decode steps. We observe that action-to-vision attention maps are highly consistent across adjacent timesteps. The top-attended visual tokens at timestep t strongly overlap with those at timestep $t - 1$, and this consecutive-decode overlap is substantially higher than the overlap between prefill attention and action-decode attention. Figure 2 further shows averaged ratios and a representative rollout.

Principle for VLA Token Pruning. Our analysis shows that VLA token pruning faces a key challenge beyond standard VLM pruning: visual tokens important for semantic prefilling do not fully cover those needed for action decoding. As a result, pruning solely based on prefill-stage semantic salience can remove action-critical tokens, explaining why directly applying VLM-oriented pruning methods degrades VLA performance. Meanwhile, the temporal continuity of action-to-vision attention provides a practical way to estimate current action-level token importance from recent decode attentions. These findings motivate a VLA-specific pruning strategy, which accounts for both semantic relevance and temporally estimated action relevance when selecting visual tokens.

4 Methodology

4.1 Problem Formulation

We formulate VLA token pruning as selecting a compact subset of visual tokens that preserves the model’s action prediction. Given visual tokens $\mathbf{E}_v$ with $|\mathbf{E}_v| = M$ and a target budget $\tilde{M} < M$, the pruning function f selects an index subset $\tilde{\mathcal{I}} \subseteq \{1, \dots, M\}$ with $|\tilde{\mathcal{I}}| = \tilde{M}$, and returns

$$\tilde{\mathbf{E}}_v = f(\mathbf{E}_v) = \{\mathbf{E}_v[i] : i \in \tilde{\mathcal{I}}\}.$$

The goal is to minimize the discrepancy between the original and pruned VLA output distributions:

$$\min_f \mathcal{L}(\mathcal{P}, \tilde{\mathcal{P}}) \quad \text{s.t.} \quad |f(\mathbf{E}_v)| = \tilde{M} \quad (5)$$

where $\mathcal{P} = P(A|\mathbf{E}_\tau, \mathbf{E}_v)$ and $\tilde{\mathcal{P}} = P(A|\mathbf{E}_\tau, f(\mathbf{E}_v))$. $P(\cdot)$ denotes the conditional generation probability of VLA model. $\mathcal{L}$ measures the difference in the model’s output with and without pruning, and $\tilde{M}$ indicates the number of retained tokens. To reflect the stage-wise structure of VLA inference, we decompose $\mathcal{P}$ into vision-language prefill P_{vl} and action decode P_{act} based on hidden states $\mathbf{Z}_\tau, \mathbf{Z}_v$:

$$\begin{aligned} \mathcal{P} &= P_{vl}(\mathbf{Z}_\tau, \mathbf{Z}_v | \mathbf{E}_\tau, \mathbf{E}_v) \cdot P_{act}(A | \mathbf{Z}_\tau, \mathbf{Z}_v), \\ \tilde{\mathcal{P}} &= \tilde{P}_{vl}(\tilde{\mathbf{Z}}_\tau, f(\tilde{\mathbf{Z}}_v) | \mathbf{E}_\tau, f(\mathbf{E}_v)) \cdot \tilde{P}_{act}(A | \tilde{\mathbf{Z}}_\tau, f(\tilde{\mathbf{Z}}_v)). \end{aligned}$$

Here, f sparsifies both the input $\mathbf{E}_v$ and corresponding hidden states $\mathbf{Z}_v$. For VLA inference, relying on single-level objectives—either semantic-level $\mathcal{L}(P_{vl}, \tilde{P}_{vl})$ or action-level $\mathcal{L}(P_{act}, \tilde{P}_{act})$ —fails to preserve the joint probability $\mathcal{P}$. To address this, we introduce VLA-Pruner.

4.2 The Proposed VLA-Pruner

VLA-Pruner consists of *semantic-action importance estimation* and a corresponding *Combine-then-Filter token selection strategy* for VLA token pruning. The overall pipeline is illustrated in Figure 3 and detailed in Algorithm 1.

4.2.1 Semantic-Action Importance Estimation

For VLMs, previous works (Chen et al., 2024; Zhang et al., 2024) have shown that prefill or text-to-vision attention scores can effectively measure visual-token importance. For VLA models, as discussed before (Sec. 3), both semantic-level and action-level visual requirements should be satisfied. Therefore, VLA-Pruner estimates token importance from two complementary perspectives: prefill attention scores $\mathcal{S}_{\text{vl}}^t$ (Eq. 2) for semantic relevance, and action-to-vision attention scores $\mathcal{S}_{\text{act}}^t$ (Eq. 3) for action-level relevance.

Attention Estimation via Temporal Smoothing.

As discussed in Sec. 3.3, action-to-vision attention exhibits short-term temporal continuity during VLA manipulation. However, the current action attention score $\mathcal{S}_{\text{act}}^t$ is unavailable when pruning decisions are made during prefilling. We therefore estimate current action-level importance from recent action-decode attentions $\{\mathcal{S}_{\text{act}}^{t-1}, \mathcal{S}_{\text{act}}^{t-2}, \dots\}$ using an Exponential Moving Average (EMA)-based temporal smoother (Hyndman and Athanasopoulos, 2018; Gardner Jr, 2006; Gardner, 1985). Specifically, since VLA manipulation depends more on short-term context (Xu et al., 2025), we use a finite-window EMA, implemented as a decaying window average over the previous w steps:

$$\hat{\mathcal{S}}_{\text{act}}^t = \frac{\sum_{i=1}^w \gamma^i \mathcal{S}_{\text{act}}^{t-i}}{\sum_{i=1}^w \gamma^i}, \quad (6)$$

where w is the window size and $\gamma \in [0, 1]$ controls the exponential decay rate. This estimator assigns larger weights to more recent action attentions while retaining short-term historical context, providing a simple proxy for current action-level visual importance. For brevity, we omit the time superscript t in the following discussion.

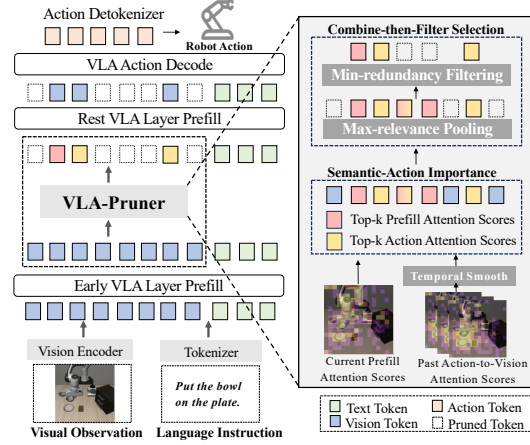


Figure 3: **Overview of VLA-Pruner** (budget $k=3$). VLA-Pruner selects semantic- and action-salient tokens, then removes redundancy to preserve compact task-relevant information.

4.2.2 Semantic-Action Token Selection via Combine-then-Filter

Given the semantic importance score $\mathcal{S}_{\text{vl}}$ and temporally estimated action importance score $\hat{\mathcal{S}}_{\text{act}}$, a straightforward approach is to normalize them, take a weighted sum, and retain top- $\tilde{M}$ tokens. However, this score-fusion scheme has several limitations. First, it introduces a sensitive weighting hyperparameter, reducing robustness and generalizability. Second, it does not explicitly reduce redundancy, so similar or jointly moderate-salience tokens may dominate the retained set (Alvar et al., 2025). Third, it relies on the estimated $\hat{\mathcal{S}}_{\text{act}}$, which can be unreliable under abrupt attention shifts.

To avoid these issues, VLA-Pruner follows a simple **Combine-then-Filter** strategy. This design follows the *minimal-Redundancy-Maximal-Relevance* (mRMR) principle (Peng et al., 2005): we first preserve tokens relevant to either inference stage and then remove redundant candidates. We organize the procedure into three phases:

(1) Semantic-Action Candidate Selection. We first select the top- $\tilde{M}$ visual tokens under the semantic and action importance scores, respectively:

$$\mathcal{C}_{\text{vl}} = \text{Top-}\tilde{M} \left(\{\mathcal{S}_{\text{vl}}[i]\}_{i=1}^M \right), \quad \mathcal{C}_{\text{act}} = \text{Top-}\tilde{M} \left(\{\hat{\mathcal{S}}_{\text{act}}[i]\}_{i=1}^M \right),$$

where $\tilde{M}$ is the target token budget. Here, $\mathcal{C}_{\text{vl}}$ contains tokens salient for semantic prefilling, while $\mathcal{C}_{\text{act}}$ contains tokens estimated to be important for action decoding.

(2) Relevance-Maximization Pooling. We then take the union of the two candidate sets:

$$\mathcal{C}_{\text{dual}} = \mathcal{C}_{\text{vl}} \cup \mathcal{C}_{\text{act}}.$$

This candidate pool preserves tokens relevant to either semantic prefilling or action decoding, maximizing two-stage relevance. Due to the attention divergence between these two stages, $|\mathcal{C}_{\text{dual}}|$ is generally larger than $\tilde{M}$.

(3) Redundancy-Minimization Filtering. Finally, we filter redundant tokens from $\mathcal{C}_{\text{dual}}$ to meet the target budget $\tilde{M}$. Following diversity-based pruning (Alvar et al., 2025), we formulate this step as a Max–Min Diversity Problem (MMDP) (Porumbel et al., 2011): selecting a subset $\tilde{\mathcal{C}} \subset \mathcal{C}_{\text{dual}}$ of size $\tilde{M}$ that maximizes the minimum pairwise distance among retained tokens:

$$\tilde{\mathcal{C}} = \underset{\mathcal{C} \subset \mathcal{C}_{\text{dual}}, |\mathcal{C}|=\tilde{M}}{\operatorname{argmax}} \min_{\substack{i,j \in \mathcal{C} \\ i \neq j}} d(v_i, v_j), \quad (7)$$

where $v_i, v_j \in \mathbf{E}_v$ are visual patch embeddings indexed by i, j , and $d(\cdot, \cdot)$ is the cosine distance:

$$d(v_i, v_j) = 1 - \frac{v_i \cdot v_j}{\|v_i\| \|v_j\|}. \quad (8)$$

This objective selects a diverse subset from $\mathcal{C}_{\text{dual}}$, thereby reducing redundancy among retained tokens. MMDP can be solved by exact or heuristic methods (Resende et al., 2010; Martí et al., 2022). To facilitate efficiency, we use a greedy algorithm that iteratively add the token whose minimum distance to the retained set is maximal, until reaching $\tilde{M}$ elements. To improve initialization, the first token is chosen by maximal second-nearest distance. The full procedure is detailed in Algorithm 2.

4.3 Implementation Details.

VLA-Pruner prunes visual tokens at Transformer layer K (K set to 3 (Chen et al., 2024)). It selects tokens using last-layer prefill attention and temporal smoothing of action-to-vision attention averaged over the latter half layers. Pruned tokens are dropped from the rest prefill layers. For decaying window average, window size w and decay rate γ are empirically set as 3 and 0.8. We warm-start VLA-Pruner with w steps to record sufficient attention history. VLA-Pruner is training-free, serving as a plug-and-play acceleration module for VLA architectures with action-to-vision cross-attention, a mechanism used by state-of-the-art VLA models (Kim et al., 2024; 2025b; Black et al., 2024), including (i) autoregressive policies (Kim et al., 2024) (e.g., OpenVLA (Kim et al., 2024)), (ii) action-chunk decoders (OpenVLA-OFT (Kim et al., 2025b)), and (iii) diffusion-head policies such as π_0 (Black et al., 2024), for which we average action attention over flow-matching steps. We provide theoretical analysis of computational complexity in Appendix A.2.

5 Experiments

We evaluate VLA-Pruner in both simulation and real-world settings. We evaluate VLA-Pruner on open-source VLA models: OpenVLA (Kim et al., 2024), OpenVLA-OFT (Kim et al., 2025b) and π_0 (Black et al., 2024). All experiments are conducted on an NVIDIA RTX 4090 GPU.

5.1 Experimental Setup

Baselines and Evaluation Protocol. We compare VLA-Pruner with a comprehensive suite of training-free accelerators under given compute budgets. Baselines include state-of-the-art token pruning methods (FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024), and DivPrune (Alvar et al., 2025)) and a VLA-specific method VLA-Cache (Xu et al., 2025). All methods are evaluated under identical budgets. To ensure a comprehensive comparison, our experiments include diverse pruning ratios (50%, 75%, 87.5%). For VLA-Cache, we set the same token reuse ratio. Our evaluation metrics mainly include: task success rate (%), inference latency (ms) and FLOPs(T). We provide more details of baselines and VLA models in Appendix B.1. EfficientVLA (Yang et al., 2025) uses token pruning strategies similar to prior work (using attention and diversity). We include results of it and HoloV (Zou et al.) in Appendix B.3.

Evaluation Benchmarks. We evaluate VLA-Pruner on simulation and real-robot benchmarks, with details in Appendix B.2. **LIBERO:** LIBERO (Liu et al., 2023a) includes four suites—*Spatial*, *Object*, *Goal*, and *Long*—covering complementary manipulation generalization settings, with 10 tasks and 500 evaluation episodes per suite. Following OpenVLA (Kim et al., 2024) and OpenVLA-OFT (Kim et al., 2025b), we use the standard evaluation protocol for direct comparison. We further evaluate the diffusion-head model π_0 (Black et al., 2024) to verify cross-architecture generality (Appendix B.4). **SIMPLER:** SIMPLER (Li et al., 2024b) provides two sim-to-real-oriented settings, visual matching (VM) and variant aggregation (VA). We report results on three tasks where OpenVLA achieves non-trivial success rates: *Move Near*, *Pick Coke Can* under VA, and *Open/Close Drawer* under VM. **Real Robot:** We evaluate VLA-Pruner on a 6-DoF xArm6 robot with a parallel gripper, using four real-world manipulation tasks (100 trials each): *Can Stack*, *Cup Pour*, *Cube Place*, and *Bottle Place*.

Table 1: Performance of VLA-Pruner on OpenVLA and OpenVLA-OFT across LIBERO at 50%, 25%, and 12.5% vision-token retention. For each method, the first line shows average success rates(%); the second line shows relative accuracy vs. vanilla (100%) or speedup ratio ($\times$). We also report FLOPs (T), inference latency (ms/action; ms/action-chunk).

Method	OpenVLA								OpenVLA-OFT							
	Spatial	Object	Goal	Long	Acc.(%) $\uparrow$	FLOPs(T) $\downarrow$	Latency(ms) $\downarrow$		Spatial	Object	Goal	Long	Acc.(%) $\uparrow$	FLOPs(T) $\downarrow$	Latency(ms) $\downarrow$	
Upper Bound (100%)																
Vanilla	87.6	84.6	78.6	52.2	100.0	1.906	236.41		95.8	98.7	96.3	90.7	100.0	3.903	135.78	
	100%	100%	100%	100%	(—)	100.00%	1.00 $\times$		100%	100%	100%	100%	(—)	1.00 $\times$		
Retain 50% Tokens ($\downarrow$ 50%)																
FastV	86.2	81.6	77.2	50.6	97.43	1.136	172.32		94.6	96.8	92.7	87.0	97.26	2.219	88.23	
	98.4%	96.5%	98.2%	96.9%	($\downarrow$ 2.57)	59.60%	1.37 $\times$		98.7%	98.1%	96.3%	95.9%	($\downarrow$ 2.74)	56.85%	1.54 $\times$	
SparseVLM	85.6	80.5	75.0	48.6	95.44	1.155	175.77		94.1	93.7	91.2	85.1	95.37	2.289	90.22	
	97.7%	95.2%	95.4%	93.1%	($\downarrow$ 4.56)	60.60%	1.34 $\times$		98.3%	94.9%	94.7%	93.8%	($\downarrow$ 4.63)	58.65%	1.50 $\times$	
DivPrune	82.6	78.8	71.8	47.6	92.64	1.105	173.88		90.8	91.1	89.9	83.1	92.61	2.126	88.01	
	94.3%	93.1%	91.3%	91.2%	($\downarrow$ 7.36)	57.97%	1.36 $\times$		94.8%	92.3%	93.3%	91.7%	($\downarrow$ 7.39)	54.47%	1.54 $\times$	
VLA-Cache	87.1	80.7	78.6	51.8	98.48	1.384	192.20		95.4	96.0	96.7	90.2	99.09	2.730	101.01	
	99.4%	95.4%	100.0%	99.2%	($\downarrow$ 1.52)	72.61%	1.23 $\times$		99.6%	97.3%	100%	99.4%	($\downarrow$ 0.91)	69.95%	1.34 $\times$	
VLA-Pruner	88.2	85.8	79.4	56.4	102.45	1.139	178.38		97.3	98.6	96.8	92.6	101.05	2.234	92.86	
	100.7%	101.4%	101.0%	108.0%	($\uparrow$ 2.45)	59.76%	1.33 $\times$		101.6%	99.9%	100.5%	102.1%	($\uparrow$ 1.05)	57.24%	1.46 $\times$	
Retain 25% Tokens ($\downarrow$ 75%)																
FastV	81.6	69.6	71.6	43.8	87.62	0.757	141.25		87.8	81.8	87.6	74.6	87.48	1.401	72.73	
	93.2%	82.3%	91.1%	83.9%	($\downarrow$ 12.38)	39.72%	1.67 $\times$		91.7%	82.9%	91.0%	82.3%	($\downarrow$ 12.52)	35.89%	1.87 $\times$	
SparseVLM	83.4	72.8	67.6	45.6	88.67	0.772	144.12		89.8	84.8	82.7	79.1	88.63	1.459	74.71	
	95.2%	86.1%	86.0%	87.4%	($\downarrow$ 11.33)	40.50%	1.64 $\times$		93.7%	85.9%	85.9%	87.2%	($\downarrow$ 11.37)	37.38%	1.82 $\times$	
DivPrune	77.4	61.3	65.4	41.4	80.96	0.743	142.12		83.9	72.6	79.3	72.5	80.76	1.389	72.30	
	88.4%	72.5%	83.2%	79.3%	($\downarrow$ 19.04)	38.98%	1.66 $\times$		87.6%	73.6%	82.3%	79.9%	($\downarrow$ 19.24)	35.58%	1.88 $\times$	
VLA-Cache	78.1	73.2	70.2	45.5	88.08	0.961	164.17		85.6	85.3	84.1	80.3	88.11	1.938	85.02	
	89.2%	86.5%	89.3%	87.2%	($\downarrow$ 11.92)	50.42%	1.44 $\times$		89.4%	86.4%	87.3%	88.5%	($\downarrow$ 11.89)	49.65%	1.60 $\times$	
VLA-Pruner	85.4	82.5	78.4	51.8	98.48	0.758	144.81		93.5	96.2	95.2	90.2	98.37	1.420	75.64	
	97.5%	97.5%	99.7%	99.2%	($\downarrow$ 1.52)	39.77%	1.63 $\times$		97.6%	97.5%	98.9%	99.5%	($\downarrow$ 1.63)	36.38%	1.80 $\times$	
Retain 12.5% Tokens ($\downarrow$ 87.5%)																
FastV	62.0	58.5	55.8	18.8	63.08	0.568	125.76		60.6	59.2	60.6	28.6	61.64	1.082	66.11	
	70.8%	69.1%	71.0%	36.0%	($\downarrow$ 36.92)	29.80%	1.88 $\times$		63.3%	60.0%	62.9%	31.5%	($\downarrow$ 38.36)	27.72%	2.05 $\times$	
SparseVLM	65.8	55.3	55.4	19.0	63.20	0.581	128.77		73.1	65.0	67.5	32.8	62.11	1.136	67.65	
	75.1%	65.4%	70.5%	36.4%	($\downarrow$ 36.80)	30.48%	1.84 $\times$		76.3%	65.8%	70.1%	36.1%	($\downarrow$ 37.89)	29.11%	2.01 $\times$	
DivPrune	55.4	54.8	51.4	17.6	58.06	0.569	125.36		61.4	61.8	64.9	30.8	56.83	1.059	66.46	
	63.2%	64.8%	65.4%	33.7%	($\downarrow$ 41.94)	29.28%	1.89 $\times$		64.1%	62.6%	67.4%	34.0%	($\downarrow$ 43.17)	27.13%	2.04 $\times$	
VLA-Cache	52.5	50.1	52.0	15.1	54.79	0.710	145.93		57.3	58.7	64.8	26.2	53.88	1.373	76.15	
	59.9%	59.2%	66.2%	28.9%	($\downarrow$ 45.21)	37.25%	1.62 $\times$		59.8%	59.5%	67.3%	28.9%	($\downarrow$ 46.12)	35.18%	1.78 $\times$	
VLA-Pruner	80.2	78.4	69.0	42.8	88.90	0.571	129.01		88.1	87.6	84.9	68.8	88.27	1.096	68.95	
	91.6%	92.7%	87.8%	82.0%	($\downarrow$ 11.10)	29.96%	1.83 $\times$		91.9%	91.7%	88.1%	81.3%	($\downarrow$ 11.73)	28.08%	1.99 $\times$	

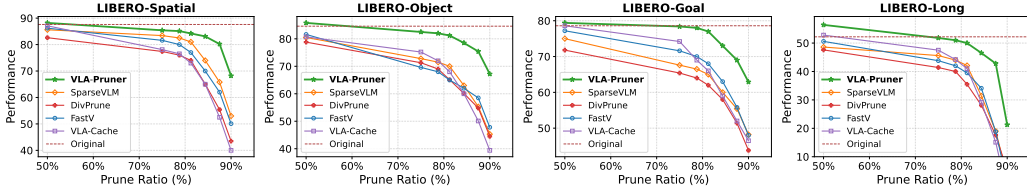


Figure 4: Performance of OpenVLA with baselines across LIBERO under varying pruning ratio.

5.2 Main Results

Results on LIBERO. We evaluate VLA-Pruner on OpenVLA (fine-tuned) and OpenVLA-OFT on LIBERO benchmark. Table 1 reports *success rate* on the four suites under token-retention ratios of 50%, 25%, and 12.5%. We report a *relative accuracy* metric, defined as the performance preservation ratio. VLA-Pruner achieves the best success rate and relative accuracy across all settings among training-free accelerators. Even at a 12.5% retention ratio, it maintains 88.9% and 88.27% relative accuracy on OpenVLA and OpenVLA-OFT, respectively, outperforming baselines by up to 34.39%. At a 50% retention ratio, VLA-Pruner can improve success rate, thanks to precise noise filtering that stabilizes policy execution. This is evident on *LIBERO-Long*, where it achieves 4.2% improvement. DivPrune degrades VLA performance since manipulation needs localized visual details. VLA-Cache achieves competitive results due to temporal awareness but fails at high compression ratios. Figure 4 visualizes more details of VLA-Pruner performance varying under different ratios.

Results on SIMPLER. We evaluate cross-environment generalization on the SIMPLER environment. Table 2 compares VLA-Pruner with acceleration baselines on OpenVLA at 75% compression ratio. VLA-Pruner best preserves accuracy, showing robust transfer across environments.

Results on Real Robot. We evaluate real-world performance on a physical 6-DoF xArm6 arm controlled by OpenVLA-OFT. Figure 5 reports results under 75% token-pruning ratio. VLA-Pruner attains the highest relative accuracy, underscoring its practicality for on-robot deployment.

Efficiency Analysis. We report inference efficiency metrics in Table 1, including inference latency and FLOPs(T). At same pruning ratios, VLA-Pruner achieves comparable speedups to visual token-pruning methods. Its latency is ~ 5 ms higher than FastV, yet it achieves significantly better performance. At the 25% retention ratio, VLA-Pruner outperforms token pruning baselines at 50%

Table 2: SIMPLER results at 75% pruning ratio.

Method	Move Near	Pick Coke Can	Open/Close Drawer	Overall
OpenVLA	54.0% (100%)	52.8% (100%)	49.5% (100%)	52.1% (100%)
FastV	38.7% (71.7%)	41.9% (79.4%)	33.7% (68.1%)	38.1% (73.1%)
VLA-Cache	41.2% (76.3%)	40.6% (76.9%)	38.8% (78.4%)	40.2% (77.2%)
VLA-Pruner	52.4% (97.0%)	50.1% (94.9%)	48.8% (98.6%)	50.4% (96.8%)

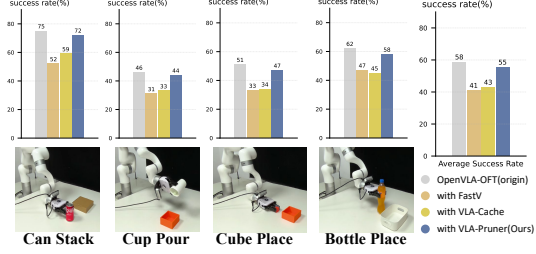


Figure 5: **Performance of VLA-Pruner on OpenVLA-OFT for real-robot tasks under a 75% pruning ratio.** We use a 6-DoF xArm6 robotic arm. VLA-Pruner better preserves model performance, demonstrating its practical advantage.

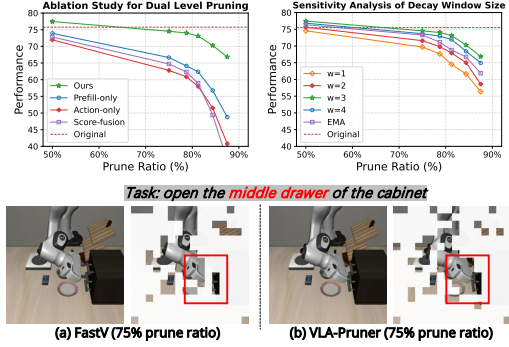


Figure 6: **Analysis of VLA-Pruner.**

retention, while being $\sim 16\%$ faster and using $\sim 20\%$ fewer FLOPs. VLA-Pruner is more compute-efficient than VLA-Cache. VLA-Pruner requires storing historical attention, but it introduces a negligible increase in Max GPU memory (see Appendix B.5.1).

5.3 Analysis of VLA-Pruner

Ablation Studies. We evaluate three alternative token selection criteria: (1) *prefill-only*, relying solely on prefill attention; (2) *action-only*, utilizing only action decode attention; and (3) *score-fusion*, a naive weighted linear combination of both. As shown in Figure 6, VLA-Pruner performs best, validating the effectiveness of our token selection strategy.

Sensitivity Analysis. We analyze the sensitivity of VLA-Pruner to the decay window size w in Figure 6, which demonstrates the robustness of our method. We further study varying decay rate γ and pruning layer K in Appendix B.5.2.

Qualitative Visualization. We provide a qualitative case study in Figure 6. VLA-Pruner maintains broad semantic understanding and preserves essential localized visual information for action.

6 Related Works

Vision-Language-Action Models (VLA) Large-scale VLMs have advanced multimodal learning by integrating visual perception with language reasoning (Li et al., 2023). Building on this progress, VLA models (Kim et al., 2024; Black et al., 2024; Kim et al., 2025b; Brohan et al., 2022; Zitkovich et al., 2023) further introduce an action modality for end-to-end motor control. They typically adopt large VLM backbones (Touvron et al., 2023), fine-tune them on robotic data (O’Neill et al., 2024), and generate actions either by discretizing them into language-like tokens, as in OpenVLA (Kim et al., 2024), or by attaching diffusion-policy heads, as in π_0 (Black et al., 2024). In both cases, fine-grained action generation commonly relies on *action-to-vision cross-attention*. Despite strong performance on robot manipulation benchmarks (Liu et al., 2023a; Li et al., 2024b), VLA models remain computationally intensive, hindering real-time deployment on resource-constrained platforms.

Visual Token Pruning for VLMs Visual token pruning (Chen et al., 2024; Zhang et al., 2024; Alvar et al., 2025; Ye et al., 2025) is a widely adopted strategy to reduce visual redundancy in VLMs. One line of these methods defines *semantic importance criteria* to rank and prune visual tokens, e.g., FastV (Chen et al., 2024) (using prefill attention), SparseVLM (Zhang et al., 2024) (using text-to-vision attention), DivPrune (Alvar et al., 2025) (using diversity-driven selection), and HoloV (Zou et al.) (using crop-wise attention and diversity). Another line of *calibration-based* approaches (e.g., FitPrune (Ye et al., 2025), VTW (Lin et al., 2025b)) select pruning layers/ratios by analyzing model outputs on a calibration set. However, existing methods either remain driven by semantic salience or require costly calibration, failing to explicitly account for the action-dependent visual requirements of VLA inference. This motivates a VLA-specific pruning strategy.

Training-Free Acceleration for VLA Models Visual token pruning has recently been integrated into training-free VLA acceleration frameworks (Yu et al., 2025). To name a few, EfficientVLA (Yu et al., 2025), SP-VLA (Li et al., 2025), and SpecPrune-VLA (Wang et al., 2025) perform structured acceleration that incorporates token pruning. However, these methods still use semantic salience to retain tokens. Thus, they rely on additional modules (e.g., diffusion-feature cache (Yang et al., 2025), lightweight generation module (Li et al., 2025), or layer-reduction (Yang et al., 2025; Wang et al., 2025)) to balance performance and speed, limiting generalizability and effectiveness. VLA-

Cache (Xu et al., 2025) exploits temporal continuity by caching static visual token features, opening a promising direction. However, its cache-based mechanism is less efficient.

7 Conclusion

We propose VLA-Pruner, a training-free token pruning method for efficient VLA inference. We reveal a stage-wise mismatch in VLA visual-token importance and addresses this by combining semantic relevance with temporally estimated action relevance. VLA-Pruner selects compact, non-redundant visual tokens through a Combine-then-Filter strategy. Across diverse manipulation tasks and multiple VLA architectures, VLA-Pruner reduces computation while preserving performance.

References

- Harsh Agrawal, Karan Desai, Yufei Wang, Xinlei Chen, Rishabh Jain, Mark Johnson, Dhruv Batra, Devi Parikh, Stefan Lee, and Peter Anderson. Nocaps: Novel object captioning at scale. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8948–8957, 2019.
- Saeed Ranjbar Alvar, Gursimran Singh, Mohammad Akbari, and Yong Zhang. Divprune: Diversity-based visual token pruning for large multimodal models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9392–9401, 2025.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A frontier large vision-language model with versatile abilities. *arXiv preprint arXiv:2308.12966*, 1(2):3, 2023.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- Can Cui, Pengxiang Ding, Wenxuan Song, Shuanghao Bai, Xinyang Tong, Zirui Ge, Runze Suo, Wanqi Zhou, Yang Liu, Bofang Jia, et al. Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation. *arXiv preprint arXiv:2505.03912*, 2025.
- Roya Firoozi, Johnathan Tucker, Stephen Tian, Anirudha Majumdar, Jiankai Sun, Weiyu Liu, Yuke Zhu, Shuran Song, Ashish Kapoor, Karol Hausman, et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, 44(5):701–739, 2025.
- Everette S. Gardner. Exponential smoothing: The state of the art. *Journal of Forecasting*, 4(1):1–28, 1985.
- Everette S Gardner Jr. Exponential smoothing: The state of the art—part ii. *International journal of forecasting*, 22(4):637–666, 2006.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- ByungOk Han, Jaehong Kim, and Jinhyeok Jang. A dual process vla: Efficient robotic manipulation leveraging vlm. *arXiv preprint arXiv:2410.15549*, 2024.

- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019.
- Rob J Hyndman and George Athanasopoulos. *Forecasting: principles and practice*. OTexts, 2018.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025a.
- Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025b.
- Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a.
- Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.
- Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024b.
- Ye Li, Yuan Meng, Zewen Sun, Kangye Ji, Chen Tang, Jiajun Fan, Xinzhu Ma, Shutao Xia, Zhi Wang, and Wenwu Zhu. Sp-vla: A joint model scheduling and token pruning approach for vla model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
- Yuncheng Li, Yale Song, Liangliang Cao, Joel Tetreault, Larry Goldberg, Alejandro Jaimes, and Jiebo Luo. Tgif: A new dataset and benchmark on animated gif description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4641–4650, 2016.
- Tao Lin, Yilei Zhong, Yuxin Du, Jingjing Zhang, Jiting Liu, Yinxinyu Chen, Encheng Gu, Ziyang Liu, Hongyi Cai, Yanwen Zou, Lixing Zou, Zhaoye Zhou, Gen Li, and Bo Zhao. Evo-1: Lightweight vision-language-action model with preserved semantic alignment, 2025a. URL <https://arxiv.org/abs/2511.04555>.
- Zhihang Lin, Mingbao Lin, Luxi Lin, and Rongrong Ji. Boosting multimodal large language models with visual tokens withdrawal for rapid inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 5334–5342, 2025b.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023a.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023b.
- Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European conference on computer vision*, pages 216–233. Springer, 2024.
- Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2024.

- Junzhu Mao, Yang Shen, Jinyang Guo, Yazhou Yao, and Xiansheng Hua. Efficient token compression for vision transformer with spatial information preserved. *arXiv preprint arXiv:2503.23455*, 2025a.
- Junzhu Mao, Yang Shen, Jinyang Guo, Yazhou Yao, Xiansheng Hua, and Hengtao Shen. Prune and merge: Efficient token compression for vision transformer with spatial information preserved. *IEEE Transactions on Multimedia*, 2025b.
- Rafael Martí, Anna Martínez-Gavara, Sergio Pérez-Peló, and Jesús Sánchez-Oro. A review on discrete diversity and dispersion maximization from an or perspective. *European Journal of Operational Research*, 299(3):795–813, 2022.
- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- Seongmin Park, Hyungmin Kim, Wonseok Jeon, Juyoung Yang, Byeongwook Jeon, Yoonseon Oh, and Jungwook Choi. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034*, 2024.
- Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on pattern analysis and machine intelligence*, 27(8):1226–1238, 2005.
- Bryan A Plummer, Liwei Wang, Chris M Cervantes, Juan C Caicedo, Julia Hockenmaier, and Svetlana Lazebnik. Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models. In *Proceedings of the IEEE international conference on computer vision*, pages 2641–2649, 2015.
- Daniel Cosmin Porumbel, Jin-Kao Hao, and Fred Glover. A simple and effective algorithm for the maxmin diversity problem. *Annals of Operations Research*, 186(1):275–293, 2011.
- Mauricio GC Resende, Rafael Martí, Micael Gallego, and Abraham Duarte. Grasp and path relinking for the max–min diversity problem. *Computers & Operations Research*, 37(3):498–508, 2010.
- Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024.
- Steven A Sloman. The empirical case for two systems of reasoning. *Psychological bulletin*, 119(1):3, 1996.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Hanzhen Wang, Jiaming Xu, Jiayi Pan, Yongkang Zhou, and Guohao Dai. Specprune-vla: Accelerating vision-language-action models via action-aware self-speculative pruning. *arXiv preprint arXiv:2509.05614*, 2025.
- Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024.
- Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 25th ACM international conference on Multimedia*, pages 1645–1653, 2017.
- Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. Vla-cache: Efficient vision-language-action manipulation via adaptive token caching. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

- Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. EfficientVLA: Training-free acceleration and compression for vision-language-action models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=SELY1DHZk2>.
- Weihao Ye, Qiong Wu, Wenhao Lin, and Yiyi Zhou. Fit and prune: Fast and training-free visual token pruning for multi-modal large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 22128–22136, 2025.
- Zhaoshu Yu, Bo Wang, Pengpeng Zeng, Haonan Zhang, Ji Zhang, Lianli Gao, Jingkuan Song, Nicu Sebe, and Heng Tao Shen. A survey on efficient vision-language-action models, 2025. URL <https://arxiv.org/abs/2510.24795>.
- Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
- Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- Xin Zou, Di Lu, Yizhou Wang, Yibo Yan, Yuanhuiyi Lyu, Xu Zheng, Linfeng Zhang, and Xuming Hu. Don’t just chase “highlighted tokens” in mllms: Revisiting visual holistic context retention. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Algorithm 1 Overall Pipeline of VLA-Pruner

```
1: Input: Visual embeddings  $\mathbf{E}_v$ , Semantic attention scores  $\mathcal{S}_{v1}$ , Action attention history  $\{\mathcal{S}_{\text{act}}^{t-i}\}_{i=1}^w$ , Token budget  $\tilde{M}$ , Decay params  $\gamma$ 
2: Output: Selected index set  $\tilde{\mathcal{C}} \subseteq \{1, \dots, M\}$ , with  $|\tilde{\mathcal{C}}| = \tilde{M}$ 
3: if  $\tilde{M} \geq M$  then
4:   return  $\{1, \dots, M\}$  ▷ Skipping pruning
5: end if
6:  $\hat{\mathcal{S}}_{\text{act}} \leftarrow \sum_{i=1}^w \gamma^i \mathcal{S}_{\text{act}}^{t-i} / \sum_{i=1}^w \gamma^i$  ▷ Action Attention Estimation via Temporal Smoothing (Eq. 6)
7:  $\mathcal{C}_{v1} \leftarrow$  indices of Top- $\tilde{M}$  elements of  $\mathcal{S}_{v1}$ 
8:  $\mathcal{C}_{\text{act}} \leftarrow$  indices of Top- $\tilde{M}$  elements of  $\hat{\mathcal{S}}_{\text{act}}$ 
9: Max-Relevance pooling:  $\mathcal{C}_{\text{dual}} \leftarrow \mathcal{C}_{v1} \cup \mathcal{C}_{\text{act}}$ 
10: Min-Redundancy filtering:  $\tilde{\mathcal{C}} \leftarrow \text{REDUNDANCY-MINIMIZATION FILTERING}(\mathbf{E}_v, \mathcal{C}_{\text{dual}}, \tilde{M})$  ▷ Alg. 2
11: return  $\tilde{\mathcal{C}}$ 
```

Algorithm 2 Redundancy-Minimization Filtering

```
1: Input: Visual embeddings  $\mathbf{E}_v = \{v_1, \dots, v_M\}$ , candidate index set  $\mathcal{C}_{\text{dual}} \subseteq \{1, \dots, M\}$ , target size  $\tilde{M}$ 
2: Output: Selected index set  $\tilde{\mathcal{C}}$ , with  $|\tilde{\mathcal{C}}| = \tilde{M}$ 
3: if  $|\mathcal{C}_{\text{dual}}| \leq \tilde{M}$  then
4:   return  $\mathcal{C}_{\text{dual}}$ 
5: end if
6: Extract candidate features:  $\{v_i\}_{i \in \mathcal{C}_{\text{dual}}}$ 
7: Normalize:  $u_i \leftarrow v_i / \|v_i\|_2$  for all  $i \in \mathcal{C}_{\text{dual}}$ 
8: Compute cosine distance matrix  $D$  over  $\mathcal{C}_{\text{dual}}$ :  $D_{ij} \leftarrow 1 - u_i^\top u_j$ 
9:  $\tilde{\mathcal{C}} \leftarrow \emptyset$  ▷ Initialize by maximal second-nearest distance
10: for each  $i \in \mathcal{C}_{\text{dual}}$  do
11:   Let  $\{d_{i1}, d_{i2}, \dots\}$  be  $D_{ij}$  for  $j \in \mathcal{C}_{\text{dual}}, j \neq i$  sorted increasingly
12:    $s_i \leftarrow d_{i2}$  ▷ Second smallest distance
13: end for
14:  $i^* \leftarrow \arg \max_{i \in \mathcal{C}_{\text{dual}}} s_i$ 
15:  $\tilde{\mathcal{C}} \leftarrow \tilde{\mathcal{C}} \cup \{i^*\}$ 
16: while  $|\tilde{\mathcal{C}}| < \tilde{M}$  do
17:   for each  $j \in \mathcal{C}_{\text{dual}} \setminus \tilde{\mathcal{C}}$  do
18:      $m_j \leftarrow \min_{i \in \tilde{\mathcal{C}}} D_{ij}$  ▷ Min distance to selected set
19:   end for
20:    $j^* \leftarrow \arg \max_{j \in \mathcal{C}_{\text{dual}} \setminus \tilde{\mathcal{C}}} m_j$ 
21:    $\tilde{\mathcal{C}} \leftarrow \tilde{\mathcal{C}} \cup \{j^*\}$ 
22: end while
23: return  $\tilde{\mathcal{C}}$ 
```

A Method Details

A.1 Algorithm Outlines

We detail VLA-Pruner in Algorithm 1 (overall pipeline) and Algorithm 2 (min-redundancy filtering).

A.2 Complexity Analysis

Following FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024) and DivPrune (Alvar et al., 2025), we estimate the computational requirement of VLA-Pruner in terms of Transformer FLOPs, and focus on how pruning visual tokens changes the effective sequence length seen by each layer. As in prior work (Alvar et al., 2025), we count the FLOPs of multi-head attention and feed-forward networks (FFNs) in the backbone (the overhead of token selection as negligible compared to the main model FLOPs.) Let T be the total number of Transformer layers in the VLA backbone and K the index of the pruning layer (we prune tokens before the K layer). Denote the numbers of text and visual tokens before pruning by N and M ($M \gg N$), and let $\mu = N + M$ be the full sequence length. After VLA-Pruner keeps a fraction ρ of the visual tokens, the sequence length becomes $\tilde{\mu} = N + \rho M$. Following (Chen et al., 2024), the FLOPs of attention and FFN for one layer with sequence length n , hidden size d , and FFN intermediate size m can be approximated as

$$C(n) = 4nd^2 + 2n^2d + 2ndm. \quad (9)$$

Without pruning, the total FLOPs are

$$\text{FLOPs}_{\text{full}} = T C(\mu). \quad (10)$$

With VLA-Pruner, the first $K - 1$ layers still use the full sequence μ , while the remaining $T - K + 1$ layers use the shortened sequence $\tilde{\mu}$, giving

$$\text{FLOPs}_{\text{prune}} = (K - 1) C(\mu) + (T - K + 1) C(\tilde{\mu}). \quad (11)$$

Thus, the theoretical FLOPs ratio is

$$r_{\text{FLOPs}} = \frac{\text{FLOPs}_{\text{prune}}}{\text{FLOPs}_{\text{full}}} = \frac{(K - 1)C(\mu) + (T - K + 1)C(\tilde{\mu})}{TC(\mu)}. \quad (12)$$

Since the attention term in $C(\cdot)$ is quadratic in sequence length, the reduction scales roughly with ρ^2 in layers after K . This matches the empirical FLOPs ratios in Tab. 1: VLA-Pruner reduces FLOPs to about 60%, 40%, and 30% of the original model at 50%, 25%, and 12.5% token retention, respectively. Temporal smoothing over a short window of action-to-vision attention has complexity $O(wM)$ per timestep, and the greedy Max-Min filtering over the dual-level candidate pool of size $O(\rho M)$ is $O((\rho M)^2)$ per prefill. In practice, these components contribute little to the total wall-clock time, as these operations are invoked only once during the prefill stage. Finally, Eq. (12) shows that VLA-Pruner offers a flexible way to trade accuracy for efficiency: the retained-token ratio $\rho \in (0, 1]$. Smaller ρ reduces FLOPs quadratically in layers after K , yielding higher speedup, while larger ρ recovers the original model’s performance. In our experiments, we report results at $\rho \in 0.5, 0.25, 0.125$, but VLA-Pruner can be used at any intermediate ratio to meet a given compute or latency target.

B Experiments Details

B.1 Setting up Details

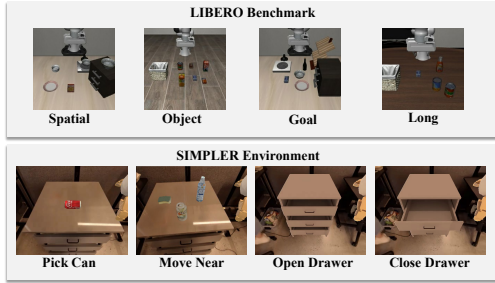
B.1.1 VLA Model Details

We consider three representative VLA architectures in our experiments: OpenVLA, OpenVLA-OFT, and π_0 . All models take RGB observations and a natural language instruction as input, and produce robot actions end-to-end.

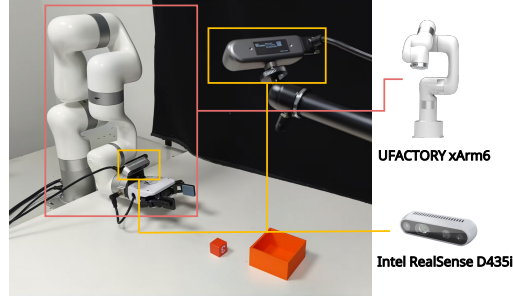
OpenVLA. OpenVLA (Kim et al., 2024) is a 7B-parameter open-source vision–language–action model built by fine-tuning a Prismatic VLM (Llama 2 language backbone with DINOv2 and SigLIP visual encoders) on robot manipulation episodes from the Open X-Embodiment dataset. The model represents actions as discrete tokens and decodes them autoregressively, enabling a single policy to control multiple embodiments out-of-the-box. OpenVLA serves as our base autoregressive VLA, and we follow its official checkpoints and evaluation settings on LIBERO.

OpenVLA-OFT. OpenVLA-OFT (Kim et al., 2025b) is an “Optimized Fine-Tuning” variant of OpenVLA that improves both control performance and inference efficiency. It augments the original decoder with parallel decoding and action chunking, and switches to a continuous action representation trained with a simple L1 regression objective. This substantially boosts success rates on LIBERO while increasing action-generation throughput. To be noted, OpenVLA-OFT utilizes bidirectional attention, meaning that during inference, there is already an action-to-vision cross-attention. However, we observe that during the early-layer prefill phase, the attention still focuses on semantic understanding, making action-to-vision less significant. In deeper layers, the action-to-vision attention starts to exhibit more distinct properties. This allows our method to effectively enhance token pruning performance by utilizing past timestep action-to-vision cross-attention at the deeper layers to guide current token pruning.

π_0 Model. π_0 (Black et al., 2024) is a prototype generalist robot policy that augments a pretrained vision–language model (PaliGemma backbone) with a continuous-action flow-matching head. Built on top of a large VLM to inherit Internet-scale semantic knowledge, π_0 is trained on diverse real-robot data and is designed for high-frequency control in dexterous manipulation tasks. The model decodes continuous motor commands via a flow-matching process with 10 integration steps. We use π_0 as a diffusion-based VLA backbone to evaluate the cross-architecture generalizability of VLA-Pruner. To capture informative interactions and reduce noise, we record the action-to-vision cross-attention scores averaged over the latter half of these integration steps.



(a) Tasks on LIBERO benchmark and the SIMPLER environment.



(b) Real-world xArm6 robot setup.

Figure 7: Evaluation environments and real-world robot setup.

B.1.2 Acceleration Baseline Details

We briefly summarize the baseline acceleration methods considered in our experiments.

FastV. FastV (Chen et al., 2024)(ECCV24) accelerates large vision-language models by pruning redundant visual tokens based on early-layer prefill attention. Its core observation is that many image patches receive consistently low attention in deeper layers while contributing substantially to computation. FastV defines an attention-based importance score during the prefill stage and drops a fixed fraction of low-score visual tokens after a chosen layer. It is training-free and purely semantic, but does not account for the action-dependent requirements of VLA models.

SparseVLM. SparseVLM (Zhang et al., 2024)(ICML25) also targets visual token redundancy in VLMs, but uses text-to-vision cross-attention as the importance signal. Tokens that receive low cross-attention from text queries are considered less relevant and pruned. Compared to FastV, SparseVLM focuses directly on aligning visual tokens with language semantics, yet still operates only at the semantic level and ignores the distinct visual needs of action decoding in VLA.

DivPrune. DivPrune (Alvar et al., 2025)(CVPR25) addresses the tendency of attention-based methods to select similar and redundant tokens. Instead of ranking tokens by attention scores, it formulates token selection as a Max-Min Diversity Problem and chooses a subset of visual tokens that are maximally dissimilar in feature space, thereby reducing redundancy while maintaining coverage. However, DivPrune is agnostic to the dual-level relevance of visual tokens in VLA prefilling and action decoding, and its diversity objective falls short for VLA tasks where localized details are critical.

VLA-Cache. VLA-Cache (Xu et al., 2025)(Neurips25) is a training-free acceleration method tailored for VLA models. It exploits temporal continuity in robotic manipulation by identifying visually static tokens whose features change little across timesteps, and reuses their computation via a KV-cache. At the same time, it uses text-to-vision cross-attention to keep task-relevant tokens fully computed. This cache-based mechanism reduces repeated computation over static background regions, but its semantic attention score based token selection remains coarse-grained, which overlooks distinct action attention distribution. To ensure a fair comparison, we set the dynamic token limit to be $1.5 \times k$ (k is the token budget), and use text-to-vision score to exclude $0.5 \times k$ of the most relevant tokens. These settings follow the default setup of VLA-Cache codebase.

EfficientVLA. EfficientVLA (Yang et al., 2025)(Neurips25) proposes a structured acceleration framework for diffusion-based VLA models that combines visual token pruning, layer reduction, and diffusion temporal feature caching. Its token-pruning component leverages semantic attention and diversity heuristics inherited from VLM pruning, while additional modules (i.e., layer skipping and diffusion caching) are used to achieve comparable acceleration. In contrast, VLA-Pruner is a single, plug-and-play token pruning module. These approaches are therefore complementary: EfficientVLA and other VLA acceleration methods can be adopted orthogonally with VLA-Pruner, while VLA-Pruner achieves strongest token-prune performance.

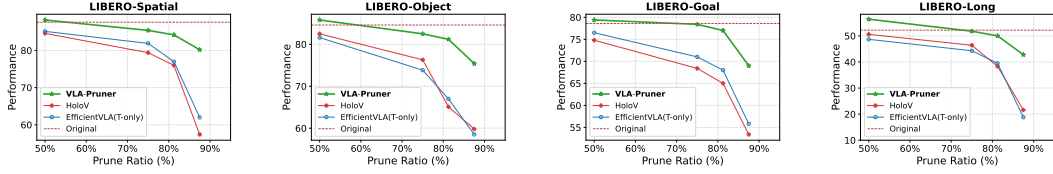


Figure 8: Performance of VLA-Pruner with more token pruning baseline methods across LIBERO tasks under varying pruning ratios. The horizontal axis represents the pruning/caching ratio of visual tokens, and the vertical axis shows the success rate.

Table 3: Performance comparison with various acceleration methods.

Method	Spatial	Object	Goal	Long	Avg.↑	FLOPs↓	Speedup↑
VLA-Pruner	85.4	82.5	78.4	51.8	74.5	39.80%	1.633×
EfficientVLA (T-only)	84.3	81.6	76.5	49.8	73.1	58.12%	1.334×
EfficientVLA	83.2	80.3	75.7	49.1	72.0	41.30%	1.491×
VTW	74.7	76.2	72.3	48.9	68.0	55.17%	1.325×
FitPrune	82.4	80.1	74.8	50.3	71.9	47.71%	1.397×

B.2 Benchmark Details

Fig. 7a shows the evaluation simulation environments and Fig. 7b presents our real-world xArm6 robot setup.

LIBERO. LIBERO (Liu et al., 2023a) is a Robosuite-based benchmark for lifelong robot manipulation that groups 130 language-conditioned tasks into four suites: *Spatial*, *Object*, *Goal*, and *Long*. LIBERO-Spatial varies only the spatial configuration of otherwise identical objects (e.g., pick up the black bowl on the stove and place it on the plate). LIBERO-Object varies the manipulated object while keeping the scene layout and goal template fixed (e.g., pick up the bbq sauce and place it in the basket). LIBERO-Goal keeps both the objects and spatial layout fixed but changes the goal description (e.g., open the middle drawer of the cabinet). LIBERO-Long corresponds to the long-horizon subset (10 tasks) of LIBERO-100, where each episode consists of multi-stage instructions such as opening a drawer, inserting an object, and then rearranging other items (e.g., put both the alphabet soup and the cream cheese box in the basket).

SIMPLER. SIMPLER (Li et al., 2024b) is a simulation suite built to closely track the real Google robot environment while allowing controlled appearance variation. It provides two evaluation settings. The *Visual Matching* (VM) setting keeps backgrounds, lighting, distractors, and camera poses close to the real setup so that rendered observations visually resemble real robot scenes and better predict real-world performance. The *Variant Aggregation* (VA) setting starts from the same base scenes but procedurally perturbs nuisance factors such as background textures, illumination, number and placement of distractor objects, table material, and camera viewpoints, creating a broad distribution shift for testing robustness. For the Google robot configuration, SIMPLER defines four canonical manipulation tasks: “pick coke can”, “pick object”, “move near”, “open/close Drawer”, and “place in closed drawer”. In our experiments, we follow the official simulator settings and success metrics, and select three representative tasks—*Pick Coke Can*, *Move Near*, and *Open/Close Drawer*—under VM or VA setting, where base model attains non-trivial performance.

Real robot. As shown in Figure 7b, our real-world system is built on a 6-DoF xArm6 manipulator equipped with two Intel RealSense D435i RGB cameras: one mounted on the wrist for close-range observations and the other fixed for capturing global scene context. The robot is controlled through the official xArmAPI. We collect real-world demonstrations following the LeRobot2.1 data specification. Each episode consists of synchronized observations recorded at 30Hz, including two RGB image streams. The proprioceptive state is represented by the robot’s absolute joint angles, and actions are stored using the same absolute joint angles to ensure consistent replay and supervision. All demonstrations are gathered using this unified multi-view setup, which provides both global scene context and fine-grained manipulation details that are essential for learning robust visuomotor policies.

Table 4: **Performance of VLA-Pruner on π_0 across LIBERO at different vision-token retention ratios.** We report success rates (%) on the four suites, relative accuracy, latency (ms), speedup ($\times$) and FLOP ratio (%).

Method	LIBERO Success (%)					Acc.(%)↑	Latency(ms)↓	Speedup (×)↑	FLOP ratio(%)↓
	Spatial	Object	Goal	Long	Avg.				
Vanilla	96.9	98.3	96.1	84.8	94.03	100.0	104.53	1.000	100
Upper Bound (100%)									
Retain 50% Tokens									
FastV	95.26	95.46	92.58	81.34	91.16	91.16	66.44	1.573	60.7
SparseVLM	94.70	93.96	91.50	79.48	89.91	89.91	70.39	1.485	61.5
DivPrune	91.30	92.14	88.25	77.19	87.22	87.22	69.17	1.511	58.9
VLA-Cache	96.47	95.99	96.59	84.65	93.43	93.43	78.30	1.335	72.3
VLA-Pruner	97.01	98.61	97.57	87.12	95.07	101.10	69.45	1.505	60.9
Retain 25% Tokens									
FastV	88.28	78.81	86.38	73.61	81.77	86.97	57.68	1.812	39.9
SparseVLM	89.63	84.78	81.80	72.67	82.22	86.76	61.09	1.711	41.3
DivPrune	84.41	71.60	80.61	66.60	75.80	80.76	59.05	1.770	39.3
VLA-Cache	85.60	85.32	85.94	73.97	82.71	87.86	64.81	1.613	51.4
VLA-Pruner	95.28	96.25	95.71	83.01	92.56	98.44	60.38	1.731	40.1
Retain 12.5% Tokens									
FastV	68.31	59.15	57.97	28.08	53.38	53.37	51.72	2.021	30.1
SparseVLM	72.04	64.98	63.55	30.99	57.89	57.89	53.79	1.943	31.3
DivPrune	63.41	63.78	62.79	29.61	54.90	54.90	52.64	1.986	29.8
VLA-Cache	59.85	58.69	64.71	27.09	52.58	55.92	59.02	1.771	39.4
VLA-Pruner	89.64	90.55	86.60	65.56	83.08	88.35	53.49	1.954	30.4

Table 5: **Memory and Runtime Analysis of Acceleration Methods on OpenVLA.** Detailed comparison of maximum GPU memory consumption and CUDA runtime across different vision-token retention rates.

Method	50% Retained		25% Retained		12.5% Retained	
	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$
Vanilla	Origin Memory: 14.75 GB / Origin CUDA Latency: 51.43 ms					
FastV	14.513	29.022	14.327	25.58	14.311	24.33
SparseVLM	14.544	29.594	14.351	25.71	14.329	23.64
DivPrune	14.438	28.052	14.303	24.98	14.281	22.91
VLA-Cache	14.592	36.33	14.523	35.13	14.484	34.57
VLA-Pruner	14.527	29.192	14.348	25.58	14.322	23.35

B.3 Results of More Baselines

Varying pruning ratio. Besides the main training-free baselines in the paper (FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024), DivPrune (Alvar et al., 2025), VLA-Cache (Xu et al., 2025)), we evaluate two recent visual token pruning methods with controllable pruning ratios: token-pruning module of EfficientVLA (Yang et al., 2025), denoted as *EfficientVLA(T-only)* and HoloV(Neurlps25) (Zou et al.). We evaluate them on OpenVLA backbone across four LIBERO suites. For each method, we report the average success rates under varying retained token ratios in Fig. 8. VLA-Pruner achieves consistent superiority across all suites. These results further support that semantic-only token pruning is ill-suited for VLA, and that our dual-level strategy better matches the requirements of VLA robotic control.

Comparison under comparable performance. We compare VLA-Pruner against structured and calibration-based baselines, focusing on the trade-off between VLA performance and computational cost. We consider the full EfficientVLA framework (Yang et al., 2025), which combines token pruning and layer reduction, as well as the calibration-based methods FitPrune (Ye et al., 2025) and VTW (Lin et al., 2025b). For EfficientVLA, we set the visual-token retention at 50% and reduce 8 layers to ensure comparable performance while maximizing acceleration. We report VLA-Pruner at 25% token retention, which achieves the better performance while requiring fewer FLOPs and lower latency. This suggests that dual-level VLA token pruning can achieve a better efficiency–performance trade-off than EfficientVLA, which is designed primarily for diffusion head policy and overlook the intrinsic characteristics of VLA inference for token pruning, thereby showing limited performance. For FitPrune and VTW, we follow their official calibration protocols and perform calibration on 100 LIBERO tasks, which corresponds to a substantially larger calibration fraction than that typically used in VLM benchmarks. Both methods achieve lower success rates than VLA-Pruner at 25% token retention and consume more FLOPs (Tab. 3). VTW typically chooses a late withdrawal layer and degrades VLA performance, while FitPrune, though effective on standard VLM benchmarks, shows limited favorable trade-off in the VLA setting since it stills rely on semantic attention scores.

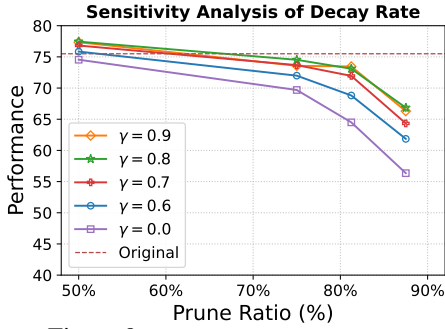


Figure 9: Ablation study for decay rate.

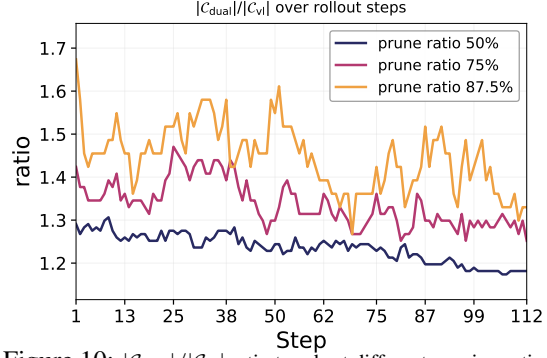


Figure 10: $|C_{\text{dual}}|/|C_{\text{vl}}|$ ratio trends at different pruning ratios on robot task.

Method	TFLOP (ratio %)	Spatial	Object	Goal	Long	Avg
K=2	37.90	69.7	74.1	55.3	34.8	58.48
K=3 (Ours)	39.77	85.4	82.5	78.4	51.8	74.53
K=4	41.98	85.5	82.3	78.1	52.0	74.48
K=5	43.08	83.9	81.7	77.9	51.4	73.73

Table 6: Study on applying pruning at different layers.

B.4 Results of π_0 on LIBERO

Setup. We evaluate VLA-Pruner on the π_0 model across LIBERO benchmark at various token retention ratios (50%, 25%, and 12.5%). For each configuration, we measure success rates, relative accuracy, latency, speedup, and FLOP ratios. We set the number of flow matching integration steps as 10, action chunk as 50 and execute 5 actions per step.

Main Results. VLA-Pruner consistently achieves high performance across all token retention settings. Notably, at each tested token retention ratio, our method maintains superior performance relative to other baselines, especially as pruning ratio increases. In particular, for π_0 , our method demonstrates impressive efficiency, maintaining comparable success rates even at 12.5% token retention. This enables a significant acceleration in computation while preserving performance, a key advantage over traditional pruning methods. While VLA-Pruner’s redundancy filtering slightly affects acceleration performance on π_0 with high-frequency, the impact remains minimal. Even at a 25% retention ratio, VLA-Pruner surpasses other token pruning methods at 50% retention ratio, demonstrating that our method can precisely preserve essential information. The results in Table 4 clearly illustrate how VLA-Pruner maintains robust task success across pruning ratios, highlighting its cross-architecture generalizability to provide an superior trade-off between computational efficiency and model accuracy for diffusion-based model.

B.5 More results

B.5.1 Detailed Efficiency Results

We report additional efficiency results, including peak GPU memory consumption (GB) and CUDA runtime (ms), across different vision-token retention ratios. As shown in Table 5, VLA-Pruner exhibits comparable efficiency to other pruning methods.

B.5.2 More Analysis

Decay rate γ . We analyze how different decay rates $\gamma \in \{0.0, 0.6, 0.7, 0.8, 0.9\}$ influence VLA-Pruner and report the results for average success rates under varying prune ratio in Fig. 9.

Pruning layer K . We compare results of different pruning layer values $K \in \{2, 3, 4, 5\}$ in Table 6 (retain 25% tokens).

VLA-Pruner details. We show $|C_{\text{dual}}|/|C_{\text{vl}}|$ ratio trends at different pruning ratios for a robotic task in Fig. 10.

B.6 More Visualization

We provide detailed visualization of VLA-Pruner token prune results across LIBERO tasks (50%, 75% pruning ratios) in Fig. 11. VLA-Pruner precisely maintains both local details for action execution and global information for semantic understanding and task planning.

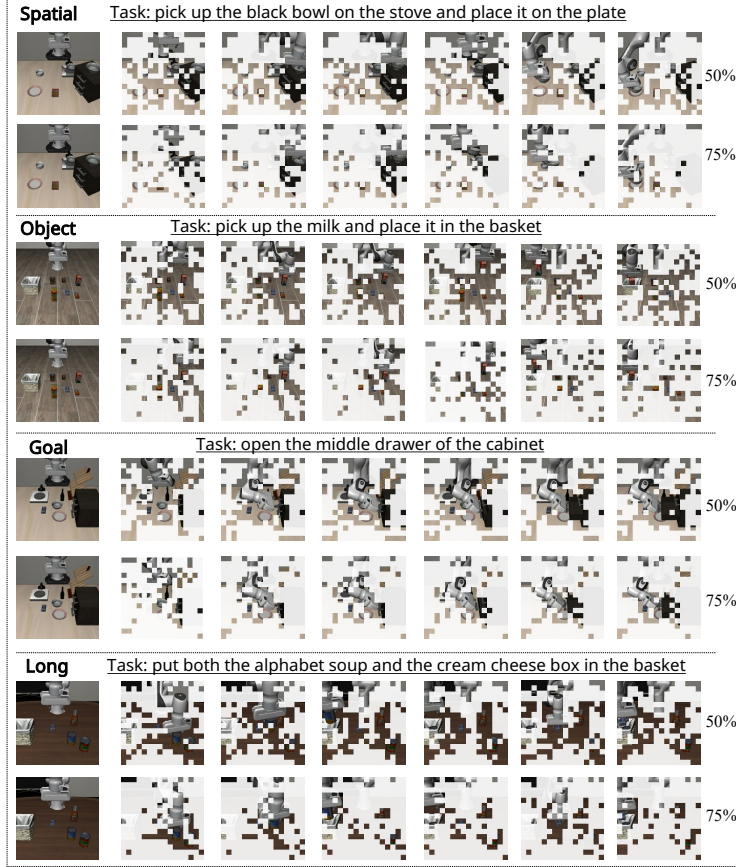


Figure 11: Visualization of VLA-Pruner token pruning results across LIBERO tasks under different ratios

C Limitations and Future work.

Our main contribution is to point out the dual-system nature of VLA inference and to propose a dual-level importance criterion that jointly accounts for semantic and action-level relevance. However, our current estimation of action-level importance relies on a heuristic temporal smoothing scheme with a fixed window and decay rate. While this works well for most tabletop settings, its benefits may diminish in highly dynamic scenarios (e.g., egocentric wrist-camera views or conveyor-belt environments), where rapid viewpoint shifts and object motion can violate the short-term temporal continuity assumption and blur sharp changes in action-to-vision attention. A promising direction for future work is to replace the fixed temporal smoothing with an adaptive prediction module that adjusts its effective window based on motion cues or feature changes (e.g., magnitude of visual dynamics or action-conditioned attention variance), or to learn a lightweight temporal attention network that directly predicts per-token importance from recent history. Such adaptive mechanisms may further improve pruning quality in dynamic environments while retaining the advantages of VLA-Pruner.